

Practical No. 1

1. Write programs to parse text files, CSV, HTML, XML and JSON documents and extract relevant data. After retrieving data check any anomalies in the data, missing values etc.
2. Write programs for reading and writing binary files
3. Write programs for searching, splitting, and replacing strings based on pattern matching using regular expressions
4. Design a relational database for a small application and populate the database. Using SQL do the CRUD (create, read, update and delete) operation

Aim:

To parse text, CSV, HTML, XML and JSON documents, extract relevant data, and check for missing values and anomalies.

Code:

```
import csv
import json
import xml.etree.ElementTree as ET
from bs4 import BeautifulSoup

# Create a text file
with open("students.txt", "w") as file:
    file.write("101, Rahul, 85\n")
    file.write("102, Priya, 92\n")
    file.write("103, Amit,\n")
    file.write("104, Sneha, 78\n")
    file.write("105, Rohan, 150\n")

# Create a CSV file
with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)

    writer.writerow(["Roll No", "Name", "Marks"])
    writer.writerow([101, "Rahul", 85])
    writer.writerow([102, "Priya", 92])
    writer.writerow([103, "Amit", ""])
    writer.writerow([104, "Sneha", 78])
    writer.writerow([105, "Rohan", 150])

# Create an HTML file
html_data = ""
<html>
<body>

<h2>Student Details</h2>

<table>
```

```
<tr>
<th>Roll No</th>
<th>Name</th>
<th>Marks</th>
</tr>
```

```
<tr>
<td>101</td>
<td>Rahul</td>
<td>85</td>
</tr>
```

```
<tr>
<td>102</td>
<td>Priya</td>
<td>92</td>
</tr>
```

```
<tr>
<td>103</td>
<td>Amit</td>
<td></td>
</tr>
```

```
<tr>
<td>104</td>
<td>Sneha</td>
<td>78</td>
</tr>
```

```
</table>
```

```
</body>
</html>
```

```
"""
```

```
with open("students.html", "w") as file:
    file.write(html_data)
```

```
# Create an XML file
root = ET.Element("students")
```

```
student = ET.SubElement(root, "student")
ET.SubElement(student, "roll_no").text = "101"
ET.SubElement(student, "name").text = "Rahul"
ET.SubElement(student, "marks").text = "85"
```

```
student = ET.SubElement(root, "student")
ET.SubElement(student, "roll_no").text = "102"
ET.SubElement(student, "name").text = "Priya"
```

```
ET.SubElement(student, "marks").text = "92"

student = ET.SubElement(root, "student")
ET.SubElement(student, "roll_no").text = "103"
ET.SubElement(student, "name").text = "Amit"
ET.SubElement(student, "marks").text = ""

tree = ET.ElementTree(root)
tree.write("students.xml")
```

Create a JSON file

```
students = [
    {
        "roll_no": 101,
        "name": "Rahul",
        "marks": 85
    },
    {
        "roll_no": 102,
        "name": "Priya",
        "marks": 92
    },
    {
        "roll_no": 103,
        "name": "Amit",
        "marks": None
    },
    {
        "roll_no": 104,
        "name": "Sneha",
        "marks": 78
    }
]
```

```
with open("students.json", "w") as file:
    json.dump(students, file, indent=4)
```

```
# -----
# 1. Parse Text File
# -----
```

```
print("\n1. TEXT FILE")
print("-----")
```

```
with open("students.txt", "r") as file:
```

```
    for line in file:
```

```
        line = line.strip()
```

```

if line == "":
    continue

data = line.split(",")

roll_no = data[0].strip()
name = data[1].strip()
marks = data[2].strip()

if marks == "":
    print("Missing marks for:", name)

else:
    marks = int(marks)

    if marks < 0 or marks > 100:
        print("Anomaly found for:", name)

    else:
        print("Roll No:", roll_no)
        print("Name:", name)
        print("Marks:", marks)
        print()

```

```

# -----
# 2. Parse CSV File
# -----

```

```

print("\n2. CSV FILE")
print("-----")

```

```

with open("students.csv", "r") as file:

```

```

    reader = csv.DictReader(file)

```

```

    for row in reader:

```

```

        name = row["Name"]
        marks = row["Marks"]

```

```

        if marks == "":
            print("Missing marks for:", name)

```

```

        else:
            marks = int(marks)

```

```

            if marks < 0 or marks > 100:
                print("Anomaly found for:", name)

```

```

        else:
            print("Name:", name)
            print("Marks:", marks)

# -----
# 3. Parse HTML File
# -----

print("\n3. HTML FILE")
print("-----")

with open("students.html", "r") as file:

    html_data = file.read()

soup = BeautifulSoup(html_data, "html.parser")

table = soup.find("table")

rows = table.find_all("tr")

for row in rows[1:]:

    columns = row.find_all("td")

    roll_no = columns[0].text.strip()
    name = columns[1].text.strip()
    marks = columns[2].text.strip()

    if marks == "":
        print("Missing marks for:", name)

    else:
        print("Roll No:", roll_no)
        print("Name:", name)
        print("Marks:", marks)

# -----
# 4. Parse XML File
# -----

print("\n4. XML FILE")
print("-----")

tree = ET.parse("students.xml")

root = tree.getroot()

for student in root.findall("student"):

```

```

roll_no = student.find("roll_no").text
name = student.find("name").text
marks_element = student.find("marks")

if marks_element.text is None or marks_element.text.strip() == "":
    print("Missing marks for:", name)

else:
    marks = int(marks_element.text)

    print("Roll No:", roll_no)
    print("Name:", name)
    print("Marks:", marks)

# -----
# 5. Parse JSON File
# -----

print("\n5. JSON FILE")
print("-----")

with open("students.json", "r") as file:

    data = json.load(file)

for student in data:

    roll_no = student["roll_no"]
    name = student["name"]
    marks = student["marks"]

    if marks is None:
        print("Missing marks for:", name)

    else:
        print("Roll No:", roll_no)
        print("Name:", name)
        print("Marks:", marks)

print("\nAll files were parsed successfully.")

```

Output:



Name: Priya

Marks: 92

Missing marks for: Amit

Roll No: 104

Name: Sneha

Marks: 78

Anomaly found for: Rohan

2. CSV FILE

Name: Rahul

Marks: 85

Name: Priya

Marks: 92

Missing marks for: Amit

Name: Sneha

Marks: 78

Anomaly found for: Rohan

3. HTML FILE

Roll No: 101

Name: Rahul

Marks: 85

Roll No: 102

Name: Priya

Marks: 92

Missing marks for: Amit

Roll No: 104

Name: Sneha

Marks: 78

Name: Rahul
Marks: 85
Roll No: 102
Name: Priya
Marks: 92
Missing marks for: Amit
Roll No: 104
Name: Sneha
Marks: 78

4. XML FILE

Roll No: 101
Name: Rahul
Marks: 85
Roll No: 102
Name: Priya
Marks: 92
Missing marks for: Amit

5. JSON FILE

Roll No: 101
Name: Rahul
Marks: 85
Roll No: 102
Name: Priya
Marks: 92
Missing marks for: Amit
Roll No: 104
Name: Sneha
Marks: 78

All files were parsed successfully.

Practical No. 2

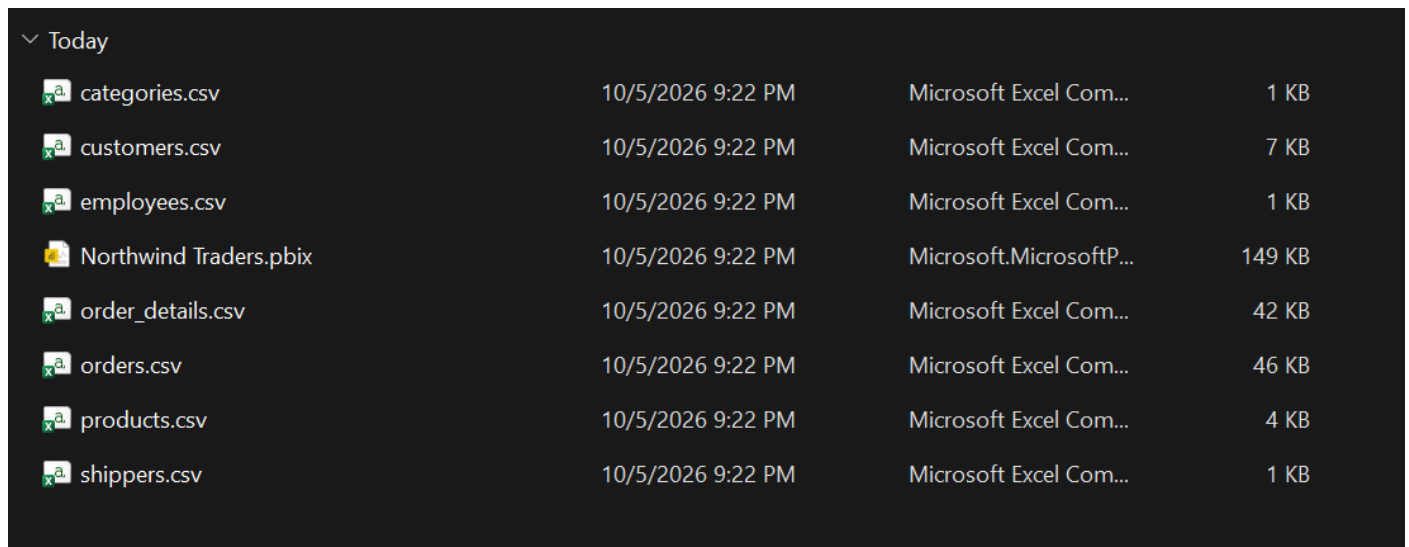
Perform the Extraction, Transformation and Loading (ETL) Process to Construct the Database in SQL Server / Power BI

Aim

To perform the **Extraction, Transformation and Loading (ETL)** process to construct a database using **SQL Server / Power BI**.

Step 1: Data Extraction

Data extraction is the first step of the ETL process. In this step, data is collected from different source systems and brought into a staging area for further processing.



The screenshot shows a file explorer window with a dark background. At the top, there is a dropdown menu labeled 'Today'. Below it, a list of files is displayed with columns for file name, date and time, file type, and size. The files listed are: categories.csv (1 KB), customers.csv (7 KB), employees.csv (1 KB), Northwind Traders.pbix (149 KB), order_details.csv (42 KB), orders.csv (46 KB), products.csv (4 KB), and shippers.csv (1 KB). All files were last modified on 10/5/2026 at 9:22 PM. Most are Microsoft Excel Comma-separated values files, while Northwind Traders.pbix is a Microsoft Power BI file.

File Name	Date and Time	File Type	Size
categories.csv	10/5/2026 9:22 PM	Microsoft Excel Comma-separated values	1 KB
customers.csv	10/5/2026 9:22 PM	Microsoft Excel Comma-separated values	7 KB
employees.csv	10/5/2026 9:22 PM	Microsoft Excel Comma-separated values	1 KB
Northwind Traders.pbix	10/5/2026 9:22 PM	Microsoft Power BI	149 KB
order_details.csv	10/5/2026 9:22 PM	Microsoft Excel Comma-separated values	42 KB
orders.csv	10/5/2026 9:22 PM	Microsoft Excel Comma-separated values	46 KB
products.csv	10/5/2026 9:22 PM	Microsoft Excel Comma-separated values	4 KB
shippers.csv	10/5/2026 9:22 PM	Microsoft Excel Comma-separated values	1 KB

There are **2 types of Data Extraction**:

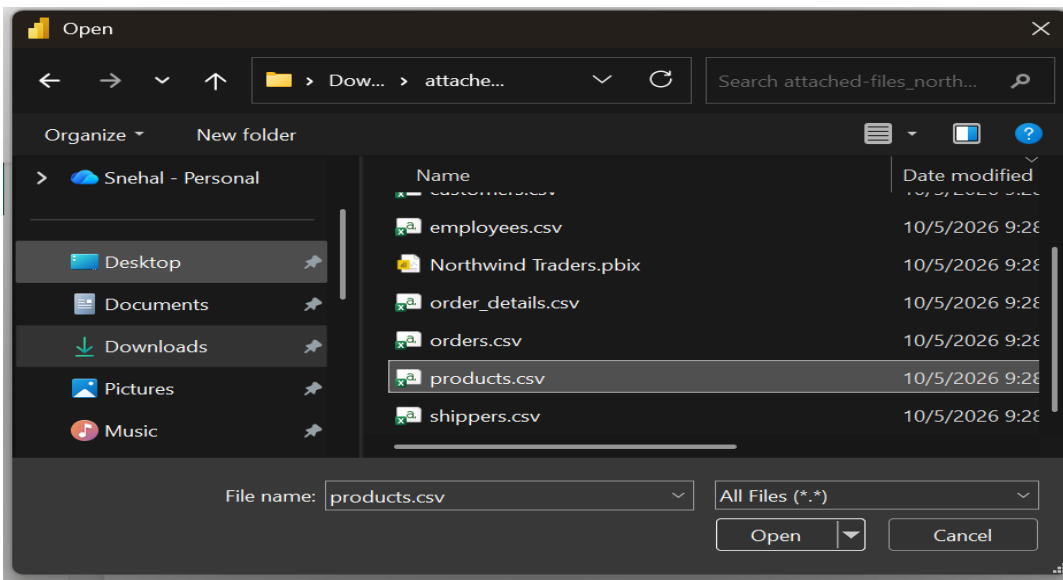
1. Full Extraction

All the data from source systems or operational systems gets extracted to the staging area.

It is generally performed during the **initial load**.

Example:

If a company has 10,000 customer records in its source database, all 10,000 records are extracted during the initial load.



2. Partial Extraction

Sometimes we receive information from the source system to update only specific or newly changed data. This is called **Partial Extraction** or **Delta Load**.

Example:

If only 200 customer records are modified today, only those 200 records are extracted instead of extracting the complete database again.

Source System Performance

The extraction strategy should not negatively affect the performance of the source system. Extraction should be planned in such a way that it uses minimum resources and does not disturb regular business operations.

Step 2: Data Transformation

Data transformation is the second step of the ETL process.

After extracting the data, there is a need to transform it according to the requirements of the target system.

The data extracted from the source system is generally in **raw format**. Therefore, it needs to be cleaned, mapped and transformed before loading it into the target server.

Important Steps of Data Transformation

1. Selection

Select only the required data that needs to be loaded into the target system.

Untitled - Power Query Editor

File Home Transform Add Column View Tools Help

Close & Apply New Source Recent Sources Enter Data Data source settings Manage Parameters Export query results Refresh Preview Properties Advanced Editor Choose Columns Remove Columns Keep Rows Remove Rows Split Column Group By Data Type: Whole Number Use First Row as Headers Replace Values Combine

Queries [2] products products (2)

fx = Table.TransformColumnTypes(#"Promoted Headers",{{"productID", Int64.Type}},

productID	productName	quantityPerUnit	unitPrice
1	Chai	10 boxes x 20 bags	
2	Chang	24 - 12 oz bottles	
3	Aniseed Syrup	12 - 550 ml bottles	
4	Chef Anton's Cajun Seasoning	48 - 6 oz jars	
5	Chef Anton's Gumbo Mix	36 boxes	
6	Grandma's Boysenberry Spread	12 - 8 oz jars	
7	Uncle Bob's Organic Dried Pears	12 - 1 lb pkgs.	
8	Northwoods Cranberry Sauce	12 - 12 oz jars	
9	Mishi Kobe Niku	18 - 500 g pkgs.	
10	Ikura	12 - 200 ml jars	
11	Queso Cabrales	1 kg pkg.	
12	Queso Manchego La Pastora	10 - 500 g pkgs.	
13	Konbu	2 kg box	
14	Tofu	40 - 100 g pkgs.	

6 COLUMNS, 77 ROWS Column profiling based on top 1000 rows

PREVIEW DOWNLOADED AT 9:42 PM

Query Settings

PROPERTIES

Name products

APPLIED STEPS

Source Promoted Headers Changed Type

2. Matching

Match the extracted data with the structure and requirements of the target system.

Untitled - Power Query Editor

File Home Transform Add Column View Tools Help

Close & Apply New Source Recent Sources Enter Data Data source settings Manage Parameters Export query results Refresh Preview Properties Advanced Editor Choose Columns Remove Columns Keep Rows Remove Rows Split Column Group By Data Type: Text Use First Row as Headers Replace Values Combine

Queries

fx = Table.PromoteHeaders(#"Removed Columns2", [PromoteAllScalars=true])

productID	productName	quantityPerUnit	unitPrice
1	Chai	10 boxes x 20 bags	18
2	Chang	24 - 12 oz bottles	19
3	Aniseed Syrup	12 - 550 ml bottles	10
4	Chef Anton's Cajun Seasoning	48 - 6 oz jars	22
5	Chef Anton's Gumbo Mix	36 boxes	21.35
6	Grandma's Boysenberry Spread	12 - 8 oz jars	25
7	Uncle Bob's Organic Dried Pears	12 - 1 lb pkgs.	30
8	Northwoods Cranberry Sauce	12 - 12 oz jars	40
9	Mishi Kobe Niku	18 - 500 g pkgs.	97
10	Ikura	12 - 200 ml jars	31
11	Queso Cabrales	1 kg pkg.	21
12	Queso Manchego La Pastora	10 - 500 g pkgs.	38
13	Konbu	2 kg box	6
14	Tofu	40 - 100 g pkgs.	23.25
15	Genen Shouyu	24 - 250 ml bottles	15.5

4 COLUMNS, 77 ROWS Column profiling based on top 1000 rows

PREVIEW DOWNLOADED AT 9:52 PM

Query Settings

PROPERTIES

Name products (2)

APPLIED STEPS

Source Removed Columns2 Promoted Headers

3. Data Transforming

Change the data according to the target table structure and business requirements.

Untitled - Power Query Editor

File Home Transform Add Column View Tools Help

Close & Apply New Source Recent Sources Enter Data Data source settings Manage Parameters Export query results Refresh Preview Properties Advanced Editor Choose Columns Remove Columns Keep Rows Remove Rows Split Column Group By Data Type: Text Use First Row as Headers Replace Values Combine

Queries

fx = Table.PromoteHeaders(#"Removed Columns2", [PromoteAllScalars=true])

productID	productName	quantityPerUnit	unitPrice
1	Chai	10 boxes x 20 bags	18
2	Chang	24 - 12 oz bottles	19
3	Aniseed Syrup	12 - 550 ml bottles	10
4	Chef Anton's Cajun Seasoning	48 - 6 oz jars	22
5	Chef Anton's Gumbo Mix	36 boxes	21.35
6	Grandma's Boysenberry Spread	12 - 8 oz jars	25
7	Uncle Bob's Organic Dried Pears	12 - 1 lb pkgs.	30
8	Northwoods Cranberry Sauce	12 - 12 oz jars	40
9	Mishi Kobe Niku	18 - 500 g pkgs.	97
10	Ikura	12 - 200 ml jars	31
11	Queso Cabrales	1 kg pkg.	21
12	Queso Manchego La Pastora	10 - 500 g pkgs.	38
13	Konbu	2 kg box	6
14	Tofu	40 - 100 g pkgs.	23.25
15	Genen Shouyu	24 - 250 ml bottles	15.5

4 COLUMNS, 77 ROWS Column profiling based on top 1000 rows

PREVIEW DOWNLOADED AT 9:52 PM

Query Settings

PROPERTIES

Name products (2)

APPLIED STEPS

Source Removed Columns2 Promoted Headers

File Home Transform Add Column View Tools Help

Group By Use First Row as Headers Count Rows

Table

Queries

productID

1 1

2 2

3 3

4 4

5 5

6 6

7 7

8 8

9 9

10 10

11 11

12 12

13 13

14 14

15 15

productID

quantityPerUnit

unitPrice

10 boxes x 20 bags

18

24 - 12 oz bottles

19

12 - 550 ml bottles

10

48 - 6 oz jars

22

36 boxes

21.35

12 - 8 oz jars

25

12 - 1 lb pkgs.

30

Northwoods Cranberry Sauce

12 - 12 oz jars

40

Mishi Kobe Niku

18 - 500 g pkgs.

97

Ikura

12 - 200 ml jars

31

Queso Cabrales

1 kg pkg.

21

Queso Manchego La Pastora

10 - 500 g pkgs.

38

Konbu

2 kg box

6

Tofu

40 - 100 g pkgs.

23.25

Genen Shouyu

24 - 250 ml bottles

15.5

4 COLUMNS, 77 ROWS Column profiling based on top 1000 rows

PREVIEW DOWNLOADED AT 9:52 PM

Query Settings

PROPERTIES

Name

products (2)

APPLIED STEPS

Source

Removed Columns2

Promoted Headers

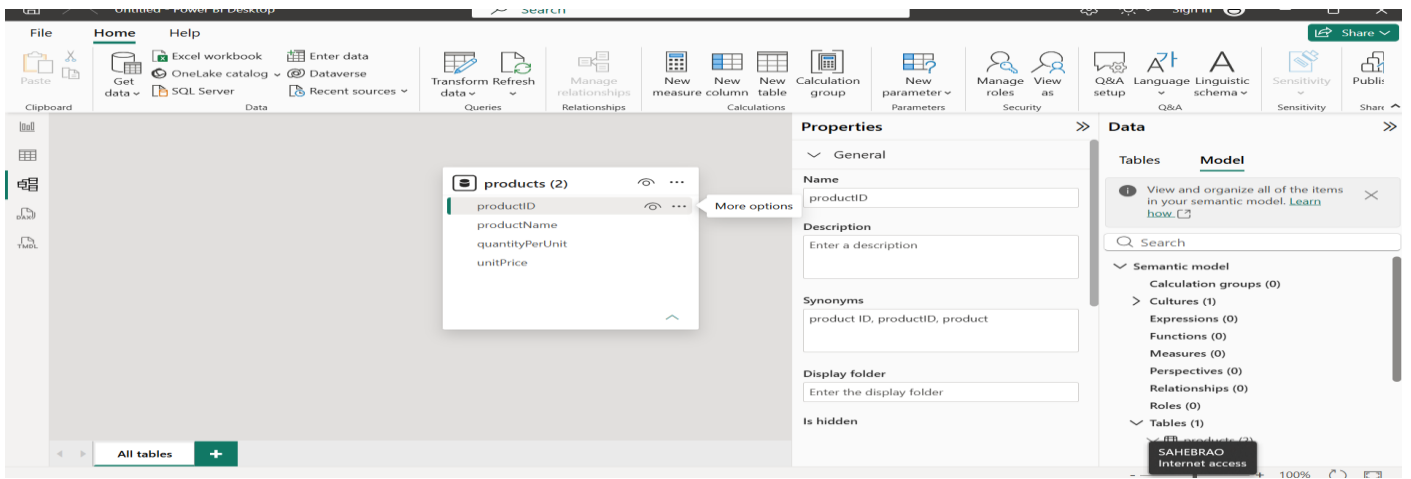
File Home Help Table tools

Name products (2)

Structure Relationships Calculations Calendars

productID productName quantityPerUnit unitPrice

productID	productName	quantityPerUnit	unitPrice
1	Chai	10 boxes x 20 bags	18
2	Chang	24 - 12 oz bottles	19
3	Aniseed Syrup	12 - 550 ml bottles	10
4	Chef Anton's Cajun Seasoning	48 - 6 oz jars	22
5	Chef Anton's Gumbo Mix	36 boxes	21.35
6	Grandma's Boysenberry Spread	12 - 8 oz jars	25
7	Uncle Bob's Organic Dried Pears	12 - 1 lb pkgs.	30
8	Northwoods Cranberry Sauce	12 - 12 oz jars	40
9	Mishi Kobe Niku	18 - 500 g pkgs.	97
10	Ikura	12 - 200 ml jars	31
11	Queso Cabrales	1 kg pkg.	21
12	Queso Manchego La Pastora	10 - 500 g pkgs.	38
13	Konbu	2 kg box	6
14	Tofu	40 - 100 g pkgs.	23.25
15	Genen Shouyu	24 - 250 ml bottles	15.5
16	Pavlova	32 - 500 g boxes	17.45
17	Alice Mutton	20 - 1 kg tins	39
18	Carnarvon Tigers	16 kg pkg.	62.5
19	Teatime Chocolate Biscuits	10 boxes x 12 pieces	9.2
20	Sir Rodney's Marmalade	30 gift boxes	81
21	Sir Rodney's Scones	24 pkgs. x 4 pieces	10



Practical No. 3

Aim:

To write a Python program to perform noise elimination, feature selection, and exploratory data analysis (EDA) on a dataset using Pandas, NumPy, Matplotlib, and Scikit-learn.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_selection import VarianceThreshold

# -----
# Sample Dataset
# -----

data = {
    "Age": [20, 21, 22, 23, 24, 100],
    "Marks": [75, 80, 85, 90, 95, 20],
    "Attendance": [85, 88, 90, 92, 94, 50],
    "Constant": [1, 1, 1, 1, 1, 1]
}

df = pd.DataFrame(data)

print("Original Dataset:")
print(df)

# -----
# 1. Noise Elimination
# Remove Outliers using IQR
# -----

Q1 = df["Age"].quantile(0.25)
Q3 = df["Age"].quantile(0.75)
```

```

IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

df_clean = df[
    (df["Age"] >= lower_limit) &
    (df["Age"] <= upper_limit)
]

print("\nDataset after Noise Elimination:")
print(df_clean)

# -----
# 2. Feature Selection
# Remove Constant Features
# -----

selector = VarianceThreshold(threshold=0.0)

selected_data = selector.fit_transform(df_clean)

selected_columns = df_clean.columns[
    selector.get_support()
]

print("\nSelected Features:")
print(list(selected_columns))

# -----
# 3. Exploratory Data Analysis
# -----

print("\nDataset Information:")
df_clean.info()

print("\nStatistical Summary:")
print(df_clean.describe())

print("\nCorrelation Matrix:")
print(df_clean.corr())

# -----
# Histogram
# -----

```

```
df_clean.hist(figsize=(8, 6))

plt.suptitle("Histogram of Features")

plt.tight_layout()

plt.show()

# -----
# Boxplot
# -----

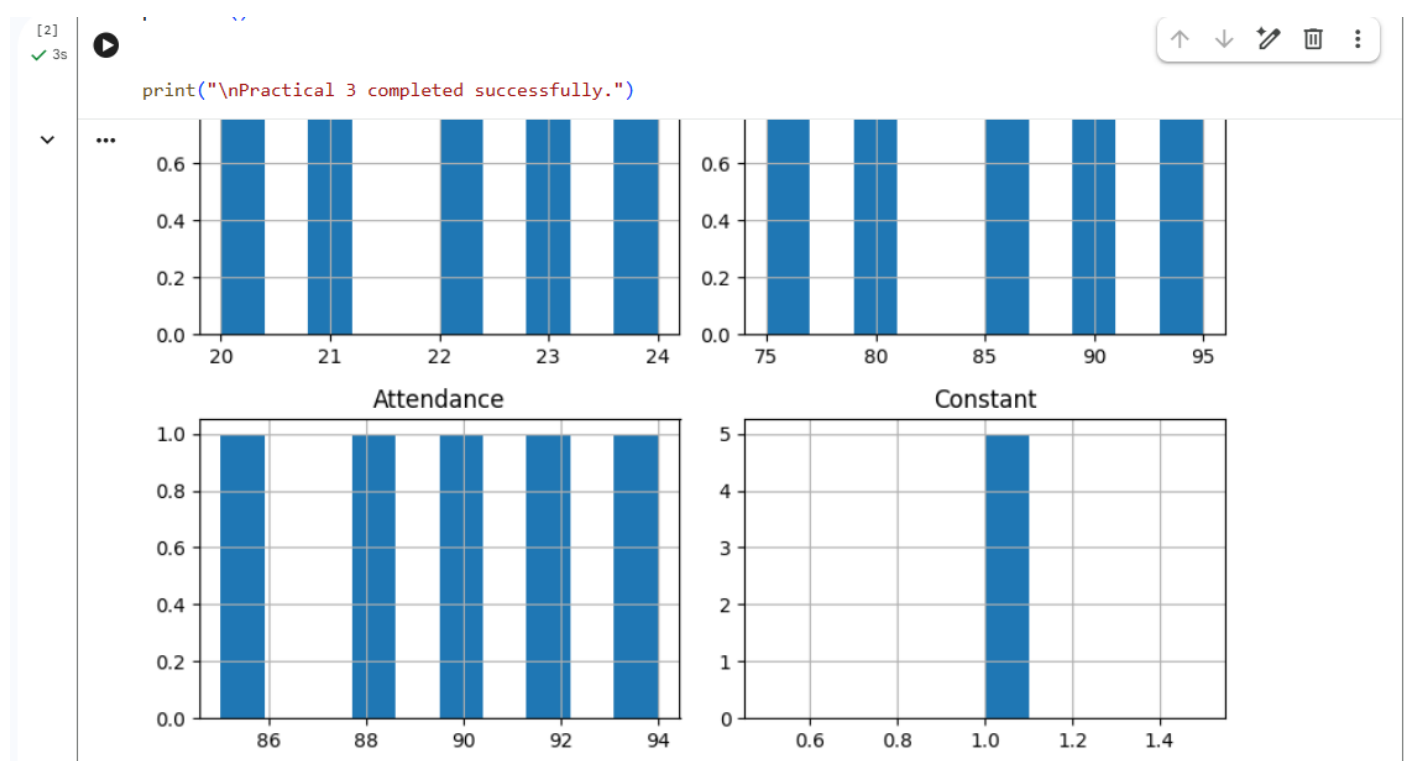
df_clean.boxplot(figsize=(8, 5))

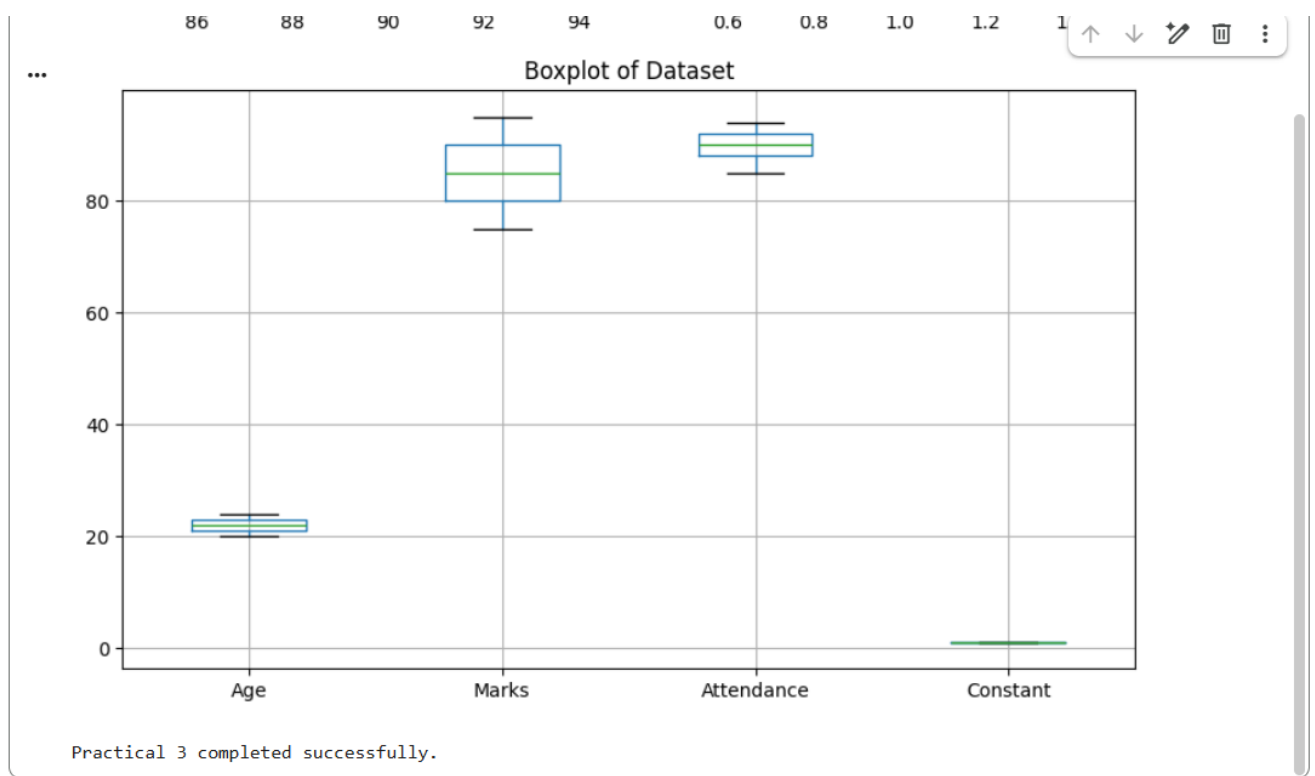
plt.title("Boxplot of Dataset")

plt.tight_layout()

plt.show()
```

Output:





Practical No. 4

Aim:

To write a Python program to extract data from a public REST API and a flat CSV file, clean and merge the extracted data, and store the final processed data in a CSV file.

Code:

```
import pandas as pd
import requests

# Function to extract data from API
def extract_api_data(url):
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()

        data = response.json()

        return pd.json_normalize(data)

    except requests.exceptions.RequestException as e:
        print("API Error:", e)

    return pd.DataFrame()
```

```

# Function to extract data from CSV file
def extract_csv_data(file_path):
    try:
        return pd.read_csv(file_path)

    except FileNotFoundError:
        print("CSV file was not found.")

    return pd.DataFrame()

# Main program
if __name__ == "__main__":

    # -----
    # 1. Extract Data from REST API
    # -----

    api_url = "https://jsonplaceholder.typicode.com/users"

    api_df = extract_api_data(api_url)

    print("API Data:")
    print(api_df[["id", "name", "email", "company.name"]])

    # -----
    # 2. Create and Read Flat CSV File
    # -----

    location_data = {
        "id": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
        "city": [
            "New York",
            "London",
            "Paris",
            "Tokyo",
            "Berlin",
            "Delhi",
            "Sydney",
            "Moscow",
            "Cairo",
            "Beijing"
        ],
        "country": [
            "USA",
            "UK",
            "France",
            "Japan",
            "Germany",

```

```

        "India",
        "Australia",
        "Russia",
        "Egypt",
        "China"
    ]
}

location_df = pd.DataFrame(location_data)

location_df.to_csv("locations.csv", index=False)

csv_df = extract_csv_data("locations.csv")

print("\nCSV Data:")
print(csv_df)


# -----
# 3. Clean API Data
# -----

api_df = api_df[[
    "id",
    "name",
    "email",
    "company.name"
]]

api_df.rename(
    columns={"company.name": "company"},
    inplace=True
)

print("\nCleaned API Data:")
print(api_df)


# -----
# 4. Merge API and CSV Data
# -----

merged_df = pd.merge(
    api_df,
    csv_df,
    on="id",
    how="inner"
)

```

```
print("\nMerged Data:")
print(merged_df)

# -----
# 5. Check Missing Values
# -----

print("\nMissing Values:")
print(merged_df.isnull().sum())

# -----
# 6. Save Final Data
# -----

merged_df.to_csv(
    "cleaned_warehouse_profiles.csv",
    index=False
)

print(
    "\nData successfully saved to "
    "'cleaned_warehouse_profiles.csv'"
)
```

Output:

[2]
✓ Os



)



...

1	2	London	UK
2	3	Paris	France
3	4	Tokyo	Japan
4	5	Berlin	Germany
5	6	Delhi	India
6	7	Sydney	Australia
7	8	Moscow	Russia
8	9	Cairo	Egypt
9	10	Beijing	China

Cleaned API Data:

	id	name	email	company
0	1	Leanne Graham	Sincere@april.biz	Romaguera-Crona
1	2	Ervin Howell	Shanna@melissa.tv	Deckow-Crist
2	3	Clementine Bauch	Nathan@yesenia.net	Romaguera-Jacobson
3	4	Patricia Lebsack	Julianne.OConner@kory.org	Robel-Corkery
4	5	Chelsey Dietrich	Lucio.Hettinger@annie.ca	Keebler LLC
5	6	Mrs. Dennis Schulist	Karley_Dach@jasper.info	Considine-Lockman
6	7	Kurtis Weissnat	Telly.Hoeger@billy.biz	Johns Group
7	8	Nicholas Runolfsdottir V	Sherwood@rosamond.me	Abernathy Group
8	9	Glenna Reichert	Chaim_McDermott@dana.io	Yost and Sons
9	10	Clementina DuBuque	Rey.Padberg@karina.biz	Hoeger LLC

Merged Data:

	id	name	email
0	1	Leanne Graham	Sincere@april.biz
1	2	Ervin Howell	Shanna@melissa.tv
2	3	Clementine Bauch	Nathan@yesenia.net
3	4	Patricia Lebsack	Julianne.OConner@kory.org
4	5	Chelsey Dietrich	Lucio_Hettinger@annie.ca
5	6	Mrs. Dennis Schulist	Karley_Dach@jasper.info
6	7	Kurtis Weissnat	Telly.Hoeger@billy.biz
7	8	Nicholas Runolfsdottir V	Sherwood@rosamond.me
8	9	Glenna Reichert	Chaim_McDermott@dana.io
9	10	Clementina DuBuque	Rey.Padberg@karina.biz

	company	city	country
0	Romaguera-Crona	New York	USA
1	Deckow-Crist	London	UK
2	Romaguera-Jacobson	Paris	France
3	Robel-Corkery	Tokyo	Japan
4	Keebler LLC	Berlin	Germany
5	Considine-Lockman	Delhi	India
6	Johns Group	Sydney	Australia
7	Abernathy Group	Moscow	Russia
8	Yost and Sons	Cairo	Egypt
9	Hoeger LLC	Beijing	China

Missing Values:

```
id      0
name    0
email   0
company 0
city    0
country 0
dtype: int64
```

Data successfully saved to 'cleaned_warehouse_profiles.csv'

Practical No. 05

Aim

To set up Apache Airflow and create a Directed Acyclic Graph (DAG) for scheduling and executing ETL tasks sequentially on a daily basis.

Step 1 — Airflow install

```
[1] pip install apache-airflow
✓ 6s
... Requirement already satisfied: aiohttp>=3.8.0 in /usr/local/lib/python3.13/dist-packages (from apache-airflow-pro
Requirement already satisfied: typing-inspection>=0.4.0 in /usr/local/lib/python3.13/dist-packages (from cadwyn>=6.1
Requirement already satisfied: cffi>=2.0.0 in /usr/local/lib/python3.13/dist-packages (from cryptography>=44.0.3->ap
Requirement already satisfied: wrapt>=1.16 in /usr/local/lib/python3.13/dist-packages (from deprecated>=1.2.13->aj
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.13/dist-packages (from fastapi<0.137.0
Requirement already satisfied: fastapi-cli>=0.0.8 in /usr/local/lib/python3.13/dist-packages (from fastapi-cli[stand
Requirement already satisfied: python-multipart>=0.0.18 in /usr/local/lib/python3.13/dist-packages (from fastapi[sta
Requirement already satisfied: email-validator>=2.0.0 in /usr/local/lib/python3.13/dist-packages (from fastapi[stand
Requirement already satisfied: pydantic-settings>=2.0.0 in /usr/local/lib/python3.13/dist-packages (from fastapi[sta
Requirement already satisfied: pydantic-extra-types>=2.0.0 in /usr/local/lib/python3.13/dist-packages (from fastapi[
Requirement already satisfied: greenlet!=0.4.17 in /usr/local/lib/python3.13/dist-packages (from greenback>=1.2.1->aj
Requirement already satisfied: sniffio in /usr/local/lib/python3.13/dist-packages (from greenback>=1.2.1->apache-air
Requirement already satisfied: outcome in /usr/local/lib/python3.13/dist-packages (from greenback>=1.2.1->apache-air
Requirement already satisfied: anyio in /usr/local/lib/python3.13/dist-packages (from httpx>=0.25.0->apache-airflow-
Requirement already satisfied: certifi in /usr/local/lib/python3.13/dist-packages (from httpx>=0.25.0->apache-airflo
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.13/dist-packages (from httpx>=0.25.0->apache-
Requirement already satisfied: idna in /usr/local/lib/python3.13/dist-packages (from httpx>=0.25.0->apache-airflow-c
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-packages (from httpcore==1.*->httpx>=0.25
Requirement already satisfied: zipp>=3.20 in /usr/local/lib/python3.13/dist-packages (from importlib-metadata>=7.0->
Requirement already satisfied: arrow>=0.15.0 in /usr/local/lib/python3.13/dist-packages (from isoduration>=20.11.0->
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages (from jinja2>=3.1.5->apach
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.13/dist-packages (from
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.13/dist-packages (from jsonschema>=4.19
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.13/dist-packages (from jsonschema>=4.19.1->
Requirement already satisfied: pyyaml-ft>=8.0.0 in /usr/local/lib/python3.13/dist-packages (from libcost>=1.8.2->apac
Requirement already satisfied: wirerope>=0.4.7 in /usr/local/lib/python3.13/dist-packages (from methodtools>=0.4.7->
Requirement already satisfied: opentelemetry-exporter-otlp-proto-grpc==1.45.0 in /usr/local/lib/python3.13/dist-pack
Requirement already satisfied: opentelemetry-exporter-otlp-proto-http==1.45.0 in /usr/local/lib/python3.13/dist-pack
```

```

Requirement already satisfied: googleapis-common-protos~=1.57 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: grpcio<2.0.0,>=1.66.2 in /usr/local/lib/python3.13/dist-packages
... Requirement already satisfied: opentelemetry-exporter-otlp-common==0.66b0 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: opentelemetry-exporter-otlp-proto-common==1.45.0 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: opentelemetry-sdk~=1.45.0 in /usr/local/lib/python3.13/dist-packages (from opentelemetry-exporter-otlp-proto-common==1.45.0)
Requirement already satisfied: protobuf<8.0,>=5.0 in /usr/local/lib/python3.13/dist-packages (from opentelemetry-exporter-otlp-proto-common==1.45.0)
Requirement already satisfied: opentelemetry-exporter-http-transport==0.66b0 in /usr/local/lib/python3.13/dist-packages (from opentelemetry-exporter-otlp-proto-common==1.45.0)
Requirement already satisfied: urllib3>=1.26 in /usr/local/lib/python3.13/dist-packages (from opentelemetry-exporter-http-transport==0.66b0)
Requirement already satisfied: tzdata>=2020.1 in /usr/local/lib/python3.13/dist-packages (from pendulum>=3.1.0->apac)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.13/dist-packages (from pydantic>=2.1)
Requirement already satisfied: pydantic-core==2.46.5 in /usr/local/lib/python3.13/dist-packages (from pydantic>=2.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.7.0->apa)
Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/python3.13/dist-packages (from python-slugify>=)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests<3,
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.13/dist-packages (from rich>=13.6.0->)
Requirement already satisfied: pathlib-abc<0.6.0,>=0.5.1 in /usr/local/lib/python3.13/dist-packages (from universal-p)
Requirement already satisfied: click>=7.0 in /usr/local/lib/python3.13/dist-packages (from uvicorn>=0.37.0->apache-a)
Requirement already satisfied: pycparser in /usr/local/lib/python3.13/dist-packages (from cffi>=2.0.0->cryptography>)
Requirement already satisfied: dnspython>=2.0.0 in /usr/local/lib/python3.13/dist-packages (from email-validator>=2.1)
Requirement already satisfied: typer>=0.16.0 in /usr/local/lib/python3.13/dist-packages (from fastapi-cli>=0.0.8->fa)
Requirement already satisfied: rich-toolkit>=0.14.8 in /usr/local/lib/python3.13/dist-packages (from fastapi-cli>=0.0.8->fa)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.13/dist-packages (from markdown-it-py>=2.2.0->ri)
Requirement already satisfied: python-dotenv>=0.21.0 in /usr/local/lib/python3.13/dist-packages (from pydantic-settings>=2.1)
Requirement already satisfied: httptools>=0.8.0 in /usr/local/lib/python3.13/dist-packages (from uvicorn[standard]>=0.37.0)
Requirement already satisfied: uvloop>=0.15.1 in /usr/local/lib/python3.13/dist-packages (from uvicorn[standard]>=0.37.0)
Requirement already satisfied: watchfiles>=0.20 in /usr/local/lib/python3.13/dist-packages (from uvicorn[standard]>=0.37.0)
Requirement already satisfied: websockets>=13.0 in /usr/local/lib/python3.13/dist-packages (from uvicorn[standard]>=0.37.0)
Requirement already satisfied: opentelemetry-semantic-conventions==0.66b0 in /usr/local/lib/python3.13/dist-packages (from opentelemetry-exporter-otlp-proto-common==1.45.0)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.13/dist-packages (from typer>=0.16.0->fa)

```

Step 2: airflow db migrate

```

[3]
✓ 11s !airflow db migrate

... 2026-10-05T16:48:10.246880Z [info] ] Performing upgrade to the metadata database [airflow.cli.commands.db_command]
2026-10-05T16:48:11.062667Z [info] ] Context impl SQLiteImpl. [alembic.runtime.migration] loc=migration.py:26
2026-10-05T16:48:11.062952Z [info] ] Will assume non-transactional DDL. [alembic.runtime.migration] loc=migration.py:26
2026-10-05T16:48:12.443660Z [info] ] Context impl SQLiteImpl. [alembic.runtime.migration] loc=migration.py:26
2026-10-05T16:48:12.443925Z [info] ] Will assume non-transactional DDL. [alembic.runtime.migration] loc=migration.py:26
2026-10-05T16:48:12.444482Z [info] ] Creating Airflow database tables from the ORM [airflow.utils.db] loc=db.py:749
2026-10-05T16:48:12.444675Z [info] ] Creating global lock context [airflow.utils.db] loc=db.py:730
2026-10-05T16:48:12.444854Z [info] ] Pool status: Pool size: 5 Connections in pool: 0 Current Overflow: -4 Current
2026-10-05T16:48:12.444974Z [info] ] Creating metadata [airflow.utils.db] loc=db.py:735
2026-10-05T16:48:13.031485Z [info] ] Getting alembic config [airflow.utils.db] loc=db.py:738
2026-10-05T16:48:13.033055Z [info] ] Stamping migration head [airflow.utils.db] loc=db.py:741
2026-10-05T16:48:13.034634Z [info] ] Context impl SQLiteImpl. [alembic.runtime.migration] loc=migration.py:26
2026-10-05T16:48:13.034854Z [info] ] Will assume non-transactional DDL. [alembic.runtime.migration] loc=migration.py:26
2026-10-05T16:48:13.099720Z [info] ] Running stamp_revision -> d2f4e1b3c5a7 [alembic.runtime.migration] loc=migration.py:26
2026-10-05T16:48:13.105893Z [info] ] Airflow database tables created [airflow.utils.db] loc=db.py:744
2026-10-05T16:48:13.331817Z [info] ] Database migration done! [airflow.cli.commands.db_command] loc=db.py:744

```

Step 3 — DAG folder

```

[4]
✓ 0s mkdir airflow_dags

[5]
✓ 0s cd airflow_dags

... /content/airflow_dags

```

Step 4 — data_extraction.py

[8]

✓ 0s



```
print("Data extraction completed successfully")
```



```
... Data extraction completed successfully
```

Step 5 — data_pipeline_dag.py

[9]

✓ 8s

```
from datetime import datetime, timedelta

from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.operators.empty import EmptyOperator

default_args = {
    "owner": "data_engineering_lab",
    "depends_on_past": False,
    "start_date": datetime(2026, 1, 1),
    "email_on_failure": False,
    "email_on_retry": False,
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    "university_etl_orchestration",
    default_args=default_args,
    description="API and Flat File ETL orchestration",
    schedule=timedelta(days=1),
    catchup=False,
) as dag:

    start_pipeline = EmptyOperator(
        task_id="start_pipeline"
```

```
[9] with DAG(
✓ 8s    "university_etl_orchestration",
        default_args=default_args,
        description="API and Flat File ETL orchestration",
        schedule=timedelta(days=1),
        catchup=False,
    ) as dag:

        start_pipeline = EmptyOperator(
            task_id="start_pipeline"
        )

        execute_extraction = BashOperator(
            task_id="run_extraction_script",
            bash_command="python data_extraction.py",
        )

        pipeline_complete = BashOperator(
            task_id="log_pipeline_success",
            bash_command='echo "ETL Execution completed successfully"',
        )

        start_pipeline >> execute_extraction >> pipeline_complete
```

▼ ... 2026-10-05T16:52:02.407352Z [warning] The `airflow.operators.bash.BashOperator` attribute is deprecated. Please use
 2026-10-05T16:52:02.412996Z [warning] The `airflow.operators.empty.EmptyOperator` attribute is deprecated. Please use

Step 6 — DAG file Airflow च्या DAGs folder

```
[11] !airflow config get-value core dags_folder
✓ 4s
```

▼ ... /root/airflow/dags

Step 7 — DAG check

```
[13] !airflow dags list
✓ 5s
```

▼ ... No data found

Step 8 — DAG structure check

```
[15] !airflow tasks list university_etl_orchestration
✓ 10s
```

▼

```
Traceback (most recent call last):
  File "/usr/local/bin/airflow", line 8, in <module>
    sys.exit(main())
    ~~~~~^
  File "/usr/local/lib/python3.13/dist-packages/airflow/__main__.py", line 61, in main
    args.func(args)
    ~~~~~^
  File "/usr/local/lib/python3.13/dist-packages/airflow/cli/cli_config.py", line 49, in command
    return func(*args, **kwargs)
  File "/usr/local/lib/python3.13/dist-packages/airflow/utils/cli.py", line 113, in wrapper
    return f(*args, **kwargs)
  File "/usr/local/lib/python3.13/dist-packages/airflow/utils/cli.py", line 464, in _wrapper
```

```

Traceback (most recent call last):
  File "/usr/local/bin/airflow", line 8, in <module>
    sys.exit(main())
    ~~~~~^
  File "/usr/local/lib/python3.13/dist-packages/airflow/__main__.py", line 61, in main
    args.func(args)
    ~~~~~^
  File "/usr/local/lib/python3.13/dist-packages/airflow/cli/cli_config.py", line 49, in command
    return func(*args, **kwargs)
  File "/usr/local/lib/python3.13/dist-packages/airflow/utils/cli.py", line 113, in wrapper
    return f(*args, **kwargs)
  File "/usr/local/lib/python3.13/dist-packages/airflow/utils/cli.py", line 464, in _wrapper
    f(*args, **kwargs)
    ~~~~~^
  File "/usr/local/lib/python3.13/dist-packages/airflow/utils/providers_configuration_loader.py", line 54, in wrapped_
    return func(*args, **kwargs)
  File "/usr/local/lib/python3.13/dist-packages/airflow/cli/commands/task_command.py", line 315, in task_list
    dag = dag or get_bagged_dag(args.bundle_name, args.dag_id)
    ~~~~~^
  File "/usr/local/lib/python3.13/dist-packages/airflow/utils/cli.py", line 304, in get_bagged_dag
    raise AirflowException(
        f"Dag {dag_id!r} could not be found; either it does not exist or it failed to parse."
    )
airflow.sdk.exceptions.AirflowException: Dag 'university_etl_orchestration' could not be found; either it does not exist

```

Step 9 — DAG run

```

[19]
✓ 7s !airflow dags trigger university_etl_orchestration

Traceback (most recent call last):
  File "/usr/local/bin/airflow", line 8, in <module>
    sys.exit(main())
    ~~~~~^
  File "/usr/local/lib/python3.13/dist-packages/airflow/__main__.py", line 61, in main
    args.func(args)
    ~~~~~^
  File "/usr/local/lib/python3.13/dist-packages/airflow/cli/cli_config.py", line 49, in command
    return func(*args, **kwargs)
  File "/usr/local/lib/python3.13/dist-packages/airflow/utils/cli.py", line 113, in wrapper
    return f(*args, **kwargs)
  File "/usr/local/lib/python3.13/dist-packages/airflow/utils/providers_configuration_loader.py", line 54, in wrapped_
    return func(*args, **kwargs)
  File "/usr/local/lib/python3.13/dist-packages/airflow/cli/commands/dag_command.py", line 95, in dag_trigger
    message = api_client.trigger_dag(
        dag_id=args.dag_id,
        ...<4 lines>...
        replace_microseconds=args.replace_microseconds,
    )
  File "/usr/local/lib/python3.13/dist-packages/airflow/api/client/local_client.py", line 47, in trigger_dag
    dag_run = trigger_dag.trigger_dag(
        dag_id=dag_id,
        ...<5 lines>...
        replace_microseconds=replace_microseconds,
    )
  File "/usr/local/lib/python3.13/dist-packages/airflow/utils/session.py", line 100, in wrapper
    return func(*args, session=session, **kwargs) # type: ignore[arg-type]
  File "/usr/local/lib/python3.13/dist-packages/airflow/api/common/trigger_dag.py", line 178, in trigger_dag
    raise DagNotFound(f"Dag id {dag_id} not found in DagModel")

```

[20]

✓ Os



```
# 1. Create the Airflow DAGs folder
!mkdir -p /root/airflow/dags

# 2. Write the extraction script to disk
with open('/root/airflow/dags/data_extraction.py', 'w') as f:
    f.write('print("Data extraction completed successfully")\n')

# 3. Write the DAG definition to the Airflow DAGs folder
with open('/root/airflow/dags/data_pipeline_dag.py', 'w') as f:
    f.write('''from datetime import datetime, timedelta
from airflow import DAG
from airflow.providers.standard.operators.bash import BashOperator
from airflow.providers.standard.operators.empty import EmptyOperator

default_args = {
    "owner": "data_engineering_lab",
    "depends_on_past": False,
    "start_date": datetime(2026, 1, 1),
    "email_on_failure": False,
    "email_on_retry": False,
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    "university_etl_orchestration",
    default_args=default_args,
    description="API and Flat File ETL orchestration",
    schedule=timedelta(days=1),
    catchup=False,
) as dag:
```

[20]

✓ Os



```
with DAG(
    "university_etl_orchestration",
    default_args=default_args,
    description="API and Flat File ETL orchestration",
    schedule=timedelta(days=1),
    catchup=False,
) as dag:

    start_pipeline = EmptyOperator(task_id="start_pipeline")

    execute_extraction = BashOperator(
        task_id="run_extraction_script",
        bash_command="python /root/airflow/dags/data_extraction.py",
    )

    pipeline_complete = BashOperator(
        task_id="log_pipeline_success",
        bash_command='echo "ETL Execution completed successfully"',
    )

    start_pipeline >> execute_extraction >> pipeline_complete
    ...

print("DAG and Extraction script successfully written to /root/airflow/dags/")
```

[21]

✓ 13s



4. Verify that Airflow now recognizes the DAG and tasks

!airflow dags list

!airflow tasks list university_etl_orchestration

v

...

train_weekly_model	.py /usr/local/lib/python3.13/dist-packages/airflow/example_dags/example_assessment_partition	airflow	True	example_dags	None
tutorial	.py /usr/local/lib/python3.13/dist-packages/airflow/example_dags/tutorial.py	airflow	True	example_dags	None
tutorial_dag	/usr/local/lib/python3.13/dist-packages/airflow/example_dags/tutorial_dag.py	airflow	True	example_dags	None
tutorial_objectstorage	/usr/local/lib/python3.13/dist-packages/airflow/example_dags/tutorial_objectstorage.py	airflow	True	example_dags	None

v

...

tutorial_task_flow_api	3/dist-packages/airflow/example_dags/tutorial_objectstorage.py /usr/local/lib/python3.13/dist-packages/airflow/example_dags/tutorial_task_flow_api.py	airflow	True	example_dags	None
tutorial_task_flow_api_virtualenv	/usr/local/lib/python3.13/dist-packages/airflow/example_dags/tutorial_task_flow_api_virtualenv.py	airflow	True	example_dags	None
tutorial_task_flow_templates	/usr/local/lib/python3.13/dist-packages/airflow/example_dags/tutorial_task_flow_templates.py	airflow	True	example_dags	None

log_pipeline_success
run_extraction_script
start_pipeline



Result: Apache Airflow was successfully configured and a daily ETL DAG was created with sequential tasks for pipeline initialization, data extraction, and successful pipeline completion logging.

Practical No. 6

Aim

To set up Apache Airflow and create a DAG to orchestrate an ETL pipeline with sequential tasks for data extraction and successful pipeline execution.

Step 1 — DAG file

```
[ ] %%writefile university_etl_orchestration.py
    # Step 1 — DAG file code goes here
```

```
▼ ... Writing university_etl_orchestration.py
```

```
[ ] schedule="@daily"
```

Step 2 — Airflow

```
[ ] !pip install apache-airflow
    !airflow db migrate

Downloading opentelemetry_sdk-1.45.0-py3-none-any.whl (140 kB)
331.1/331.1 kB 15.2 MB/s eta 0:00:00
Downloading opentelemetry_api-1.45.0-py3-none-any.whl (60 kB)
140.1/140.1 kB 9.9 MB/s eta 0:00:00
Downloading opentelemetry_semantic_conventions-0.66b0-py3-none-any.whl (206 kB)
60.0/60.0 kB 3.7 MB/s eta 0:00:00
206.3/206.3 kB 16.2 MB/s eta 0:00:00
Downloading rich_toolkit-0.20.5-py3-none-any.whl (39 kB)
Installing collected packages: lockfile, wirerope, uuid6, svcs, structlog, setproctitle, pyyaml-ft, python-daemon,
Attempting uninstall: psutil
  Found existing installation: psutil 5.9.5
  Uninstalling psutil-5.9.5:
    Successfully uninstalled psutil-5.9.5
Attempting uninstall: opentelemetry-api
  Found existing installation: opentelemetry-api 1.42.1
  Uninstalling opentelemetry-api-1.42.1:
    Successfully uninstalled opentelemetry-api-1.42.1
Attempting uninstall: opentelemetry-semantic-conventions
  Found existing installation: opentelemetry-semantic-conventions 0.63b1
  Uninstalling opentelemetry-semantic-conventions-0.63b1:
    Successfully uninstalled opentelemetry-semantic-conventions-0.63b1
Attempting uninstall: opentelemetry-sdk
  Found existing installation: opentelemetry-sdk 1.42.1
  Uninstalling opentelemetry-sdk-1.42.1:
    Successfully uninstalled opentelemetry-sdk-1.42.1
Attempting uninstall: fastapi
...
206.3/206.3 kB 16.2 MB/s eta 0:00:00
... Downloading rich_toolkit-0.20.5-py3-none-any.whl (39 kB)
Installing collected packages: lockfile, wirerope, uuid6, svcs, structlog, setproctitle, pyyaml-ft, python-daemon, py
Attempting uninstall: psutil
  Found existing installation: psutil 5.9.5
  Uninstalling psutil-5.9.5:
    Successfully uninstalled psutil-5.9.5
Attempting uninstall: opentelemetry-api
  Found existing installation: opentelemetry-api 1.42.1
  Uninstalling opentelemetry-api-1.42.1:
    Successfully uninstalled opentelemetry-api-1.42.1
Attempting uninstall: opentelemetry-semantic-conventions
  Found existing installation: opentelemetry-semantic-conventions 0.63b1
  Uninstalling opentelemetry-semantic-conventions-0.63b1:
    Successfully uninstalled opentelemetry-semantic-conventions-0.63b1
Attempting uninstall: opentelemetry-sdk
  Found existing installation: opentelemetry-sdk 1.42.1
  Uninstalling opentelemetry-sdk-1.42.1:
    Successfully uninstalled opentelemetry-sdk-1.42.1
Attempting uninstall: fastapi
  Found existing installation: fastapi 0.141.1
  Uninstalling fastapi-0.141.1:
    Successfully uninstalled fastapi-0.141.1
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behav
google-adk 2.7.1 requires opentelemetry-api<=1.42.1,>=1.39, but you have opentelemetry-api 1.45.0 which is incompatib
google-adk 2.7.1 requires opentelemetry-sdk<=1.42.1,>=1.39, but you have opentelemetry-sdk 1.45.0 which is incompatib
Successfully installed a2wsgi-1.10.10aiosmtplib-5.1.3alembic-1.20.0apache-airflow-3.3.2apache-airflow-core-3.3.2
WARNING: The following packages were previously imported in this runtime:
[psutil]
You must restart the runtime in order to use newly installed versions.
```

RESTART SESSION

You must restart the runtime in order to use newly installed versions.

RESTART SESSION

```
2026-10-05T17:12:33.760747Z [info] ] Performing upgrade to the metadata database [airflow.cli.commands.db_command
2026-10-05T17:12:34.513732Z [info] ] Context impl SQLiteImpl. [alembic.runtime.migration] loc=migration.py:7
2026-10-05T17:12:34.514066Z [info] ] Will assume non-transactional DDL. [alembic.runtime.migration] loc=migration.
2026-10-05T17:12:35.777570Z [info] ] Context impl SQLiteImpl. [alembic.runtime.migration] loc=migration.py:7
2026-10-05T17:12:35.777865Z [info] ] Will assume non-transactional DDL. [alembic.runtime.migration] loc=migration.
2026-10-05T17:12:35.778419Z [info] ] Creating Airflow database tables from the ORM [airflow.utils.db] loc=db.py:74
2026-10-05T17:12:35.778622Z [info] ] Creating global lock context [airflow.utils.db] loc=db.py:730
2026-10-05T17:12:35.778858Z [info] ] Pool status: Pool size: 5 Connections in pool: 0 Current Overflow: -4 Curren
2026-10-05T17:12:35.779052Z [info] ] Creating metadata [airflow.utils.db] loc=db.py:735
2026-10-05T17:12:36.361178Z [info] ] Getting alembic config [airflow.utils.db] loc=db.py:738
2026-10-05T17:12:36.362576Z [info] ] Stamping migration head [airflow.utils.db] loc=db.py:741
2026-10-05T17:12:36.363946Z [info] ] Context impl SQLiteImpl. [alembic.runtime.migration] loc=migration.py:7
2026-10-05T17:12:36.364142Z [info] ] Will assume non-transactional DDL. [alembic.runtime.migration] loc=migration.
2026-10-05T17:12:36.423403Z [info] ] Running stamp_revision -> d2f4e1b3c5a7 [alembic.runtime.migration] loc=migre
2026-10-05T17:12:36.429789Z [info] ] Airflow database tables created [airflow.utils.db] loc=db.py:744
2026-10-05T17:12:36.628876Z [info] ] Database migration done! [airflow.cli.commands.db_command] loc=db_comme
```

Step 3 — DAG check

```
[3]
✓ 7s !airflow dags list | grep university_etl_orchestration
```

```
[4]
✓ 10s !airflow dags test university_etl_orchestration 2026-10-05
```

```
2026-10-05T17:39:53.370514Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_
2026-10-05T17:39:53.370915Z [info] ] setting next dagrun info [airflow.models.dag] loc=dag.py:848 next_dagru
2026-10-05T17:39:53.372934Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_
2026-10-05T17:39:53.373183Z [info] ] setting next dagrun info [airflow.models.dag] loc=dag.py:848 next_dagru
2026-10-05T17:39:53.374576Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_
2026-10-05T17:39:53.374810Z [info] ] setting next dagrun info [airflow.models.dag] loc=dag.py:848 next_dagru
2026-10-05T17:39:53.376434Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_
2026-10-05T17:39:53.376656Z [info] ] setting next dagrun info [airflow.models.dag] loc=dag.py:848 next_dagru
2026-10-05T17:39:53.379233Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_
2026-10-05T17:39:53.379483Z [info] ] setting next dagrun info [airflow.models.dag] loc=dag.py:848 next_dagru
2026-10-05T17:39:53.382011Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_
2026-10-05T17:39:53.382239Z [info] ] setting next dagrun info [airflow.models.dag] loc=dag.py:848 next_dagru
2026-10-05T17:39:53.383403Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_
2026-10-05T17:39:53.383615Z [info] ] setting next dagrun info [airflow.models.dag] loc=dag.py:848 next_dagru
2026-10-05T17:39:53.386408Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_
2026-10-05T17:39:53.386636Z [info] ] setting next dagrun info [airflow.models.dag] loc=dag.py:848 next_dagru
2026-10-05T17:39:53.388728Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_
2026-10-05T17:39:53.388963Z [info] ] setting next dagrun info [airflow.models.dag] loc=dag.py:848 next_dagru
2026-10-05T17:39:53.391100Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_
2026-10-05T17:39:53.391317Z [info] ] setting next dagrun info [airflow.models.dag] loc=dag.py:848 next_dagru
2026-10-05T17:39:53.395493Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_
2026-10-05T17:39:53.395845Z [info] ] setting next dagrun info [airflow.models.dag] loc=dag.py:848 next_dagru
2026-10-05T17:39:53.410156Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_
2026-10-05T17:39:53.410592Z [info] ] setting next dagrun info [airflow.models.dag] loc=dag.py:848 next_dagru
2026-10-05T17:39:53.416322Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_
```

```

2026-10-05T17:39:53.410156Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_i
2026-10-05T17:39:53.410592Z [info] ] setting next dagrun info [airflow.models.dag] loc=dag.py:848 next_dagru
2026-10-05T17:39:53.416322Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_i
2026-10-05T17:39:53.416697Z [info] ] setting next dagrun info [airflow.models.dag] loc=dag.py:848 next_dagru
Traceback (most recent call last):
  File "/usr/local/bin/airflow", line 8, in <module>
    sys.exit(main())
    ~~~~~^
  File "/usr/local/lib/python3.13/dist-packages/airflow/__main__.py", line 61, in main
    args.func(args)
    ~~~~~^
  File "/usr/local/lib/python3.13/dist-packages/airflow/cli/cli_config.py", line 49, in command
    return func(*args, **kwargs)
  File "/usr/local/lib/python3.13/dist-packages/airflow/utils/cli.py", line 113, in wrapper
    return f(*args, **kwargs)
  File "/usr/local/lib/python3.13/dist-packages/airflow/utils/providers_configuration_loader.py", line 54, in wrapper
    return func(*args, **kwargs)
  File "/usr/local/lib/python3.13/dist-packages/airflow/utils/session.py", line 100, in wrapper
    return func(*args, session=session, **kwargs) # type: ignore[arg-type]
  File "/usr/local/lib/python3.13/dist-packages/airflow/cli/commands/dag_command.py", line 830, in dag_test
    dag = dag or get_bagged_dag(
    ~~~~~^
    bundle_names=args.bundle_name,
    ~~~~~^
    dag_id=args.dag_id,
    ~~~~~^
    dagfile_path=args.dagfile_path,
    ~~~~~^
    )
    ^
  File "/usr/local/lib/python3.13/dist-packages/airflow/utils/cli.py", line 304, in get_bagged_dag
    raise AirflowException(
        f"Dag {dag_id!r} could not be found; either it does not exist or it failed to parse."
    )
airflow.sdk.exceptions.AirflowException: Dag 'university_etl_orchestration' could not be found; either it does not exist or it failed to parse.

```

DAG task list

```

[6]
✓ 8s !airflow tasks list university_etl_orchestration

Traceback (most recent call last):
  File "/usr/local/bin/airflow", line 8, in <module>
    sys.exit(main())
    ~~~~~^
  File "/usr/local/lib/python3.13/dist-packages/airflow/__main__.py", line 61, in main
    args.func(args)
    ~~~~~^
  File "/usr/local/lib/python3.13/dist-packages/airflow/cli/cli_config.py", line 49, in command
    return func(*args, **kwargs)
  File "/usr/local/lib/python3.13/dist-packages/airflow/utils/cli.py", line 113, in wrapper
    return f(*args, **kwargs)
  File "/usr/local/lib/python3.13/dist-packages/airflow/utils/cli.py", line 464, in _wrapper
    f(*args, **kwargs)
    ~~~~~^
  File "/usr/local/lib/python3.13/dist-packages/airflow/utils/providers_configuration_loader.py", line 54, in wrapped
    return func(*args, **kwargs)
  File "/usr/local/lib/python3.13/dist-packages/airflow/cli/commands/task_command.py", line 315, in task_list
    dag = dag or get_bagged_dag(args.bundle_name, args.dag_id)
    ~~~~~^
  File "/usr/local/lib/python3.13/dist-packages/airflow/utils/cli.py", line 304, in get_bagged_dag
    raise AirflowException(
        f"Dag {dag_id!r} could not be found; either it does not exist or it failed to parse."
    )
airflow.sdk.exceptions.AirflowException: Dag 'university_etl_orchestration' could not be found; either it does not exist or it failed to parse.

```

Result

The Apache Airflow DAG was successfully created and executed. The ETL pipeline tasks were orchestrated sequentially, and the pipeline completed successfully.

Practical No. 7

Aim

To implement incremental data loading in an ETL pipeline instead of loading the entire dataset every time.

Objective

To load only new records into the target database, reducing processing time and avoiding duplicate data.

Python Code

```
import pandas as pd
import sqlite3
import os

# Step 1: Read CSV file
file_path = "sales.csv"

if not os.path.exists(file_path):
    data = pd.DataFrame({
        "id": [1, 2, 3, 4, 5],
        "product": ["Laptop", "Mouse", "Keyboard", "Monitor", "Printer"],
        "amount": [50000, 800, 1500, 12000, 7000]
    })
    data.to_csv(file_path, index=False)
```

```

df = pd.read_csv(file_path)

# Step 2: Connect to database
conn = sqlite3.connect("sales.db")

# Step 3: Create target table if it does not exist
conn.execute("""
CREATE TABLE IF NOT EXISTS sales (
    id INTEGER PRIMARY KEY,
    product TEXT,
    amount REAL
)
""")

# Step 4: Find the last loaded ID
result = pd.read_sql(
    "SELECT MAX(id) AS last_id FROM sales",
    conn
)

last_id = result["last_id"].iloc[0]

if pd.isna(last_id):
    last_id = 0

# Step 5: Select only new records
new_data = df[df["id"] > last_id]

# Step 6: Load new records
if not new_data.empty:
    new_data.to_sql(
        "sales",
        conn,
        if_exists="append",
        index=False
    )
    print("New records loaded successfully.")
else:
    print("No new records found.")

# Step 7: Display final data
final_data = pd.read_sql("SELECT * FROM sales", conn)

print("\nData in Database:")
print(final_data)

```

```
conn.close()
```

Output:

```
conn.close()
```

New records loaded successfully.

Data in Database:

	id	product	amount
0	1	Laptop	50000.0
1	2	Mouse	800.0
2	3	Keyboard	1500.0
3	4	Monitor	12000.0
4	5	Printer	7000.0

Result The ETL pipeline successfully performs incremental loading by identifying the last loaded record and inserting only new records into the database. This avoids reloading the complete dataset and prevents duplicate records.

Practical No. 8

Aim

To perform basic data processing operations using PySpark DataFrames, including reading CSV data, displaying schema, filtering, grouping, aggregation, removing duplicate records, joining datasets, and calculating average sales by product category.

Requirements

- Python
- PySpark
- CSV dataset

Program

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import avg, sum, count

# Create Spark Session
spark = SparkSession.builder \
    .appName("PySpark Practical 8") \
    .getOrCreate()
```

```

# -----
# 1. Read CSV Dataset
# -----

df = spark.read.csv(
    "sales.csv",
    header=True,
    inferSchema=True
)

print("Original Dataset:")
df.show()

# -----
# 2. Display Schema
# -----

print("Dataset Schema:")
df.printSchema()

# -----
# 3. Filtering
# -----

print("Filtered Records:")
df.filter(df.Sales > 5000).show()

# -----
# 4. Grouping and Aggregation
# -----

print("Total Sales by Category:")
df.groupBy("Category") \
    .agg(sum("Sales").alias("Total_Sales")) \
    .show()

# -----
# 5. Remove Duplicate Records
# -----

df_unique = df.dropDuplicates()

print("Dataset After Removing Duplicates:")
df_unique.show()

# -----

```

```

# 6. Read Second Dataset
# -----

customer_df = spark.read.csv(
    "customers.csv",
    header=True,
    inferSchema=True
)

print("Customer Dataset:")
customer_df.show()

# -----
# 7. Join Two Datasets
# -----

joined_df = df.join(
    customer_df,
    df.Customer_ID == customer_df.Customer_ID,
    "inner"
)

print("Joined Dataset:")
joined_df.show()

# -----
# 8. Average Sales by Product Category
# -----

print("Average Sales by Product Category:")

df.groupBy("Category") \
    .agg(avg("Sales").alias("Average_Sales")) \
    .show()

# Stop Spark
spark.stop()

```

Output:

Filtered Records:

Customer_ID	Category	Sales
101	Electronics	6000
101	Electronics	6000

Total Sales by Category:

Category	Total_Sales
Electronics	13500
Office Supplies	800
Furniture	3000

Dataset After Removing Duplicates:

Customer_ID	Category	Sales
103	Electronics	1500
101	Electronics	6000
102	Furniture	3000
104	Office Supplies	800

Dataset After Removing Duplicates:

Customer_ID	Category	Sales
103	Electronics	1500
101	Electronics	6000
102	Furniture	3000
104	Office Supplies	800

Customer Dataset:

Customer_ID	Customer_Name
101	Alice
102	Bob
103	Charlie
104	David

Joined Dataset:

Customer_ID	Category	Sales	Customer_ID	Customer_Name
101	Electronics	6000	101	Alice
102	Furniture	3000	102	Bob
103	Electronics	1500	103	Charlie
101	Electronics	6000	101	Alice
104	Office Supplies	800	104	David

Average Sales by Product Category:

Category	Average_Sales
Electronics	4500.0
Office Supplies	800.0
Furniture	3000.0

Result The PySpark program successfully:

1. Read the CSV dataset.
2. Displayed the dataset schema.
3. Performed filtering operations.
4. Performed grouping and aggregation.
5. Removed duplicate records.
6. Joined two datasets using Customer_ID.
7. Calculated average sales for each product category. Note: Keep sales.csv and customers.csv in the same folder as your PySpark program, or provide their complete file paths.



Practical No. 9

Aim

To build an end-to-end ETL pipeline for an e-commerce dataset using Python/Pandas and SQL, including data cleaning, loading into a SQL database, report generation, and handling both historical and newly arriving data.

Objectives

1. Convert CSV data into a structured dataset using Pandas.
2. Clean invalid, missing, and duplicate records.
3. Store customers, products, orders, and payments in a SQL database.
4. Generate a simple analytical report.
5. Handle both historical data and newly arriving data incrementally.

Python Program

```
import pandas as pd
import sqlite3
import os

# -----
# 1. Create / Read E-commerce Data
# -----

customers = pd.DataFrame({
    "customer_id": [1, 2, 3, 4],
    "customer_name": ["Rahul", "Priya", "Amit", "Sneha"],
    "city": ["Mumbai", "Pune", "Thane", "Nashik"]
})

products = pd.DataFrame({
    "product_id": [101, 102, 103, 104],
    "product_name": ["Laptop", "Mobile", "Headphones", "Keyboard"],
    "category": ["Electronics", "Electronics", "Accessories", "Accessories"],
    "price": [55000, 25000, 1500, 1200]
})

orders = pd.DataFrame({
    "order_id": [1001, 1002, 1003, 1004],
    "customer_id": [1, 2, 3, 4],
    "product_id": [101, 102, 103, 104],
    "quantity": [1, 2, 3, 2],
    "order_date": [
        "2026-09-01",
```

```

    "2026-09-02",
    "2026-09-03",
    "2026-09-04"
]
})

payments = pd.DataFrame({
    "payment_id": [501, 502, 503, 504],
    "order_id": [1001, 1002, 1003, 1004],
    "payment_method": ["UPI", "Card", "Cash", "UPI"],
    "payment_status": ["Completed", "Completed", "Completed", "Pending"]
})

# -----
# 2. Data Cleaning
# -----

# Remove duplicate records
customers = customers.drop_duplicates()
products = products.drop_duplicates()
orders = orders.drop_duplicates()
payments = payments.drop_duplicates()

# Handle missing values
customers["city"] = customers["city"].fillna("Unknown")
products["price"] = products["price"].fillna(0)

# Convert date into proper format
orders["order_date"] = pd.to_datetime(orders["order_date"])

# Remove invalid prices
products = products[products["price"] > 0]

print("Data Cleaning Completed")

# -----
# 3. Transformation
# -----

# Merge orders with product information
order_details = orders.merge(
    products,
    on="product_id",
    how="left"
)

```

```

# Calculate total amount
order_details["total_amount"] = (
    order_details["quantity"] * order_details["price"]
)

print("\nOrder Details:")
print(order_details)

# -----
# 4. Connect to SQL Database
# -----

connection = sqlite3.connect("ecommerce.db")

# Load data into SQL tables
customers.to_sql(
    "customers",
    connection,
    if_exists="replace",
    index=False
)

products.to_sql(
    "products",
    connection,
    if_exists="replace",
    index=False
)

orders.to_sql(
    "orders",
    connection,
    if_exists="replace",
    index=False
)

payments.to_sql(
    "payments",
    connection,
    if_exists="replace",
    index=False
)

order_details.to_sql(
    "order_details",

```

```

    connection,
    if_exists="replace",
    index=False
)

print("\nData Loaded Successfully into SQL Database")

# -----
# 5. Generate Report
# -----

report = pd.read_sql_query("""
SELECT
    p.category,
    SUM(od.quantity) AS total_quantity,
    SUM(od.total_amount) AS total_sales
FROM order_details od
JOIN products p
ON od.product_id = p.product_id
GROUP BY p.category
ORDER BY total_sales DESC
""", connection)

print("\n===== SALES REPORT =====")
print(report)

# -----
# 6. Historical + New Data
# -----

# Newly arriving order
new_order = pd.DataFrame({
    "order_id": [1005],
    "customer_id": [1],
    "product_id": [102],
    "quantity": [1],
    "order_date": [pd.Timestamp("2026-09-05")]
})

# Check whether order already exists
existing_ids = pd.read_sql_query(
    "SELECT order_id FROM orders",
    connection
)

```

```

new_order = new_order[
    ~new_order["order_id"].isin(existing_ids["order_id"])
]

# Insert only new records
if not new_order.empty:
    new_order.to_sql(
        "orders",
        connection,
        if_exists="append",
        index=False
    )

print("\nNew Data Added Successfully")

# Close database connection
connection.close()

print("\nETL Pipeline Completed Successfully!")

```

Output:

```

[1] connection.close()
✓ Os print("\nETL Pipeline Completed Successfully!")

```

▼ ... Data Cleaning Completed

Order Details:

	order_id	customer_id	product_id	quantity	order_date	product_name	\
0	1001	1	101	1	2026-09-01	Laptop	
1	1002	2	102	2	2026-09-02	Mobile	
2	1003	3	103	3	2026-09-03	Headphones	
3	1004	4	104	2	2026-09-04	Keyboard	

	category	price	total_amount
0	Electronics	55000	55000
1	Electronics	25000	50000
2	Accessories	1500	4500
3	Accessories	1200	2400

Data Loaded Successfully into SQL Database

===== SALES REPORT =====

	category	total_quantity	total_sales
0	Electronics	3	105000
1	Accessories	5	6900

New Data Added Successfully

ETL Pipeline Completed Successfully!

Practical No. 10

Aim

To design and implement an end-to-end data engineering solution including data ingestion, data cleaning, data transformation, data storage, data warehouse design, and reporting.

Mini Project Title

RideFlow – Smart Ride Data Engineering Platform

Project Description

RideFlow is an end-to-end data engineering project designed to process ride-booking data. The project demonstrates the complete data engineering workflow, starting from data ingestion and cleaning to transformation, database storage, data warehouse design, and interactive reporting.

Technologies Used

- Python
- Pandas
- SQL
- Streamlit
- Plotly
- CSV Dataset

1. Data Ingestion

The ride-booking dataset is collected in CSV format and loaded into Python using Pandas.

2. Data Cleaning

The dataset is cleaned by:

- Removing duplicate records
- Handling missing values

- Correcting data types
- Removing invalid records
- Standardizing column values

3. Data Transformation

The cleaned data is transformed to generate useful information such as:

- Total bookings
- Total revenue
- Average ride distance
- Booking status
- Payment method analysis
- Vehicle/category-wise analysis

4. Data Storage

The processed data is stored in a structured database so that it can be queried and analyzed efficiently.

5. Data Warehouse Design

The project uses a structured warehouse-style design with fact and dimension data for analytical reporting.

Fact Table: Ride/Booking details

Dimension Tables:

- Customer Dimension
- Driver Dimension
- Vehicle Dimension
- Date Dimension
- Payment Dimension

6. Reporting and Dashboard

An interactive dashboard is created using Streamlit and Plotly. It displays important ride-related insights using charts, tables, and summary information.

Project Repository

The complete project source code is available on GitHub:

RideFlow GitHub Repository:

<https://github.com/SnehalPatil26/RideFlow>

Mini Project Link:

<https://rideflow-2.onrender.com>

Output:





RideFlow

SMART RIDE DATA PLATFORM

Overview

Ride Analytics

Vehicle Intelligence

Location Intelligence

Data Pipeline

About

Vehicle Intelligence

Vehicle-level ride performance

LIVE DATA

Vehicle Intelligence

Vehicle-level ride performance generated from the processed RideFlow SQLite dataset.

Vehicle Type	Rides	Avg Booking Value	Avg Distance
Prime Sedan	14,877	₹557.81	15.76 km
eBike	14,816	₹552.21	15.58 km
Auto	14,755	₹548.44	6.24 km
Prime Plus	14,707	₹547.42	15.45 km
Bike	14,662	₹544.75	15.53 km
Prime SUV	14,655	₹541.34	15.27 km

RideFlow

SMART RIDE DATA PLATFORM

Overview

Ride Analytics

Vehicle Intelligence

Location Intelligence

Data Pipeline

About

Location Intelligence

Pickup and drop location insights

LIVE DATA

Location Intelligence

RideFlow uses pickup and drop location information from the cleaned ride-booking dataset.

Top Pickup Locations

Pickup Location	Rides
Banashankari	2,201
Yeshwanthpur	2,139
RT Nagar	2,135
Indiranagar	2,133
Sahakar Nagar	2,126

Top Drop Locations

Drop Location	Rides
Peenya	2,159
Mysore Road	2,148
MG Road	2,128
Hennur	2,120
HSR Layout	2,117

RideFlow

SMART RIDE DATA PLATFORM

Overview

Ride Analytics

Vehicle Intelligence

Location Intelligence

Data Pipeline

About

Data Pipeline

End-to-end RideFlow data engineering process

LIVE DATA

Data Engineering Pipeline

RideFlow processes raw ride-booking data through ingestion, cleaning, validation, transformation and database storage.

01
Data Ingestion

02
Data Cleaning

03
Validation

04
Transformation

05
SQLite Storage

RAW RECORDS
103,024

DATABASE
SQLite

PIPELINE STATUS
SUCCESS

DATA SOURCE
CSV

RideFlow

SMART RIDE DATA PLATFORM

Overview

Ride Analytics

Vehicle Intelligence

Location Intelligence

Data Pipeline

About

About RideFlow

Smart Ride Data Engineering & Analytics Platform

LIVE DATA

About RideFlow

RideFlow is a Smart Ride Data Engineering and Analytics Platform designed to demonstrate an end-to-end data pipeline using real ride-booking data.

The system performs data ingestion, cleaning, validation, transformation and storage before exposing analytics through a Flask API and interactive dashboard.

Python

Pipeline development and data processing.

Pandas

Data cleaning and transformation.

SQLite

Processed data storage.

Flask

Analytics API layer.

Chart.js

Interactive analytics visualization.

HTML / CSS / JS

Interactive dashboard interface.